

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스 소개

DAY1	DAY2	DAY3	DAY4	DAY5
데이터 분석을 위한 기초 수학			데이터 집계 및 시각화	
DAY6	DAY7	DAY8	DAY9	DAY10
	미니 프로젝트	확률분포, AB테스트		
DAY11	DAY12	DAY13	DAY14	DAY15
머신러닝				
DAY16	DAY17	DAY18	DAY19	DAY20
딥러닝		파이널 프로젝트		

□ 보팅(voting)

□ 배깅(bagging)

□ 부스팅(boosting)

□ 스택킹(stackng)

AI/BigData 인텐시브 과정

Section 1. 보팅

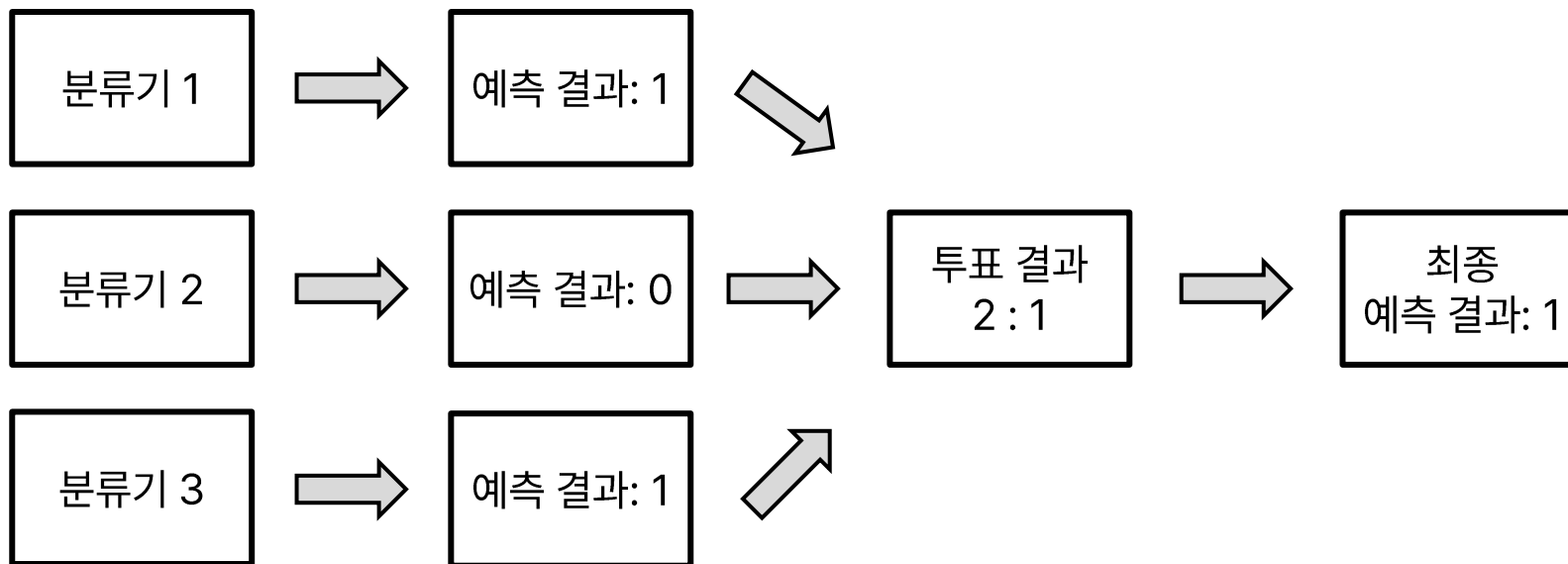
Section 1-1. 보팅개념

앙상블 학습(ensemble learning)

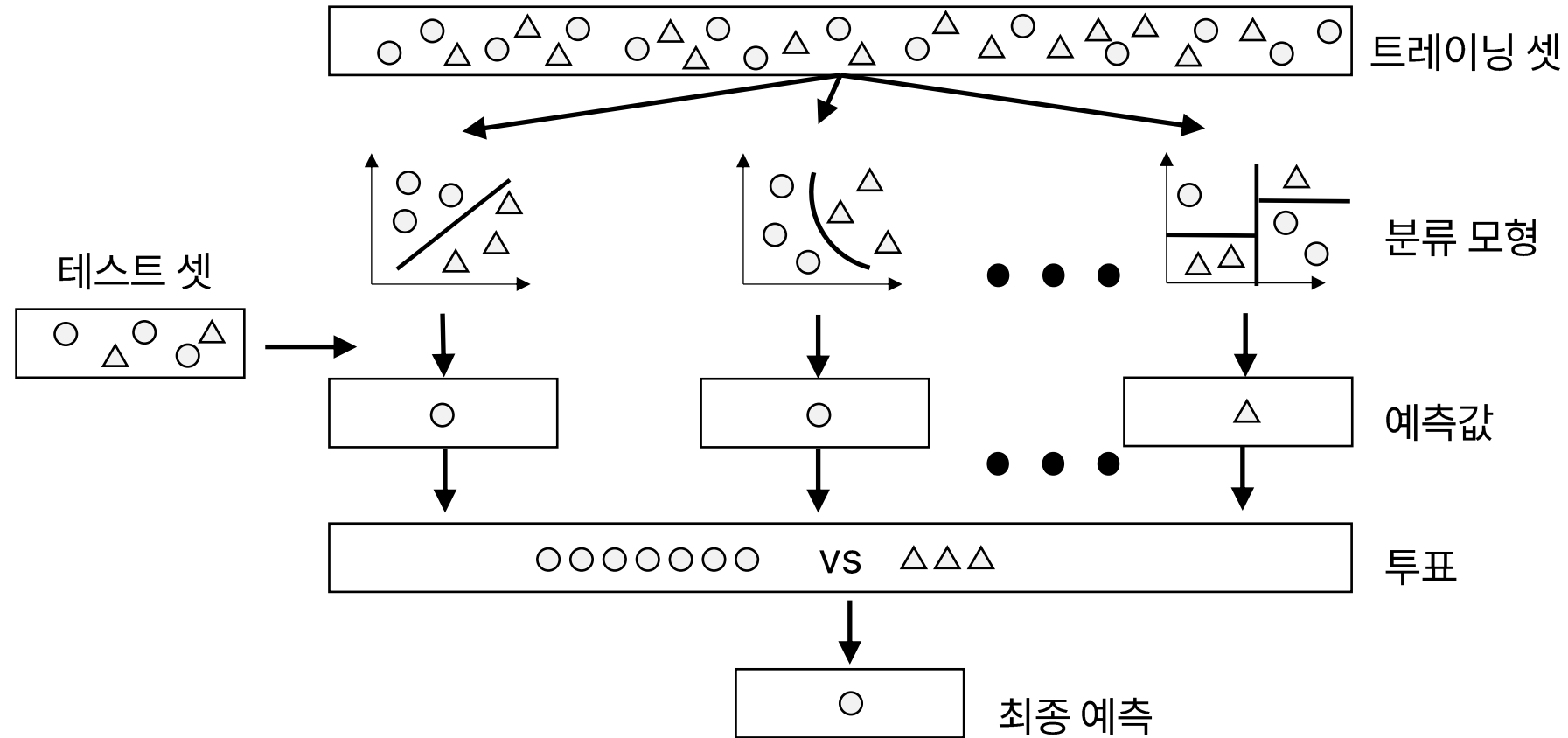
- 여러 가지 지도학습 알고리즘을 조합해서 데이터를 학습시킴
- 트레이닝 데이터를 활용한 분류 모형을 여러 개 만들고 비교
- 개별 분류 모형을 분류기(classifier)라고 함
- 성능이 낮은 분류기를 조합해 성능 좋은 분류기 생성

보팅(voting)

➤ 여러 가지 지도학습 알고리즘의 결과를 투표를 통해 최종 라벨 지정

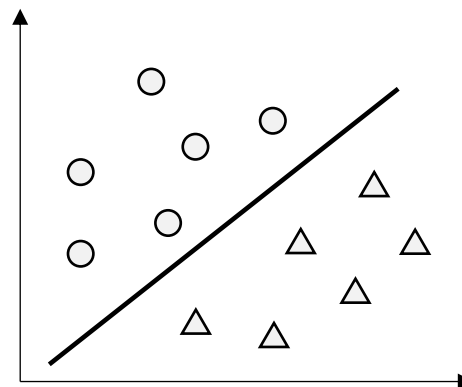


보팅(voting)



보팅(voting)

학습

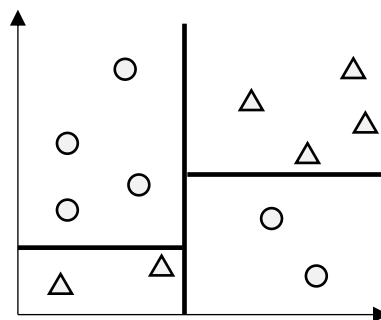
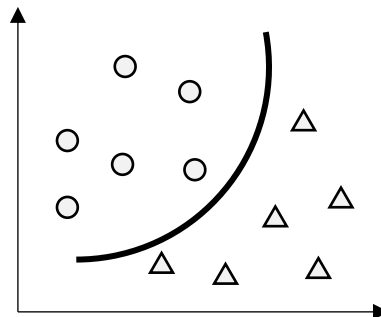
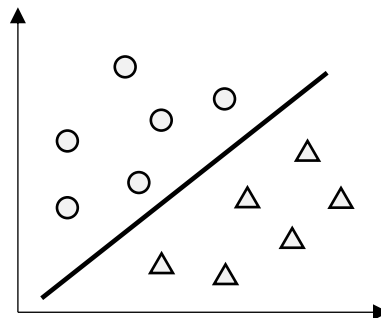


예측

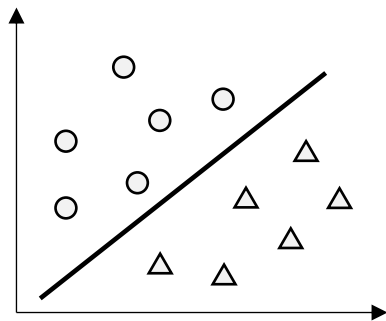
○
△

보팅(voting)

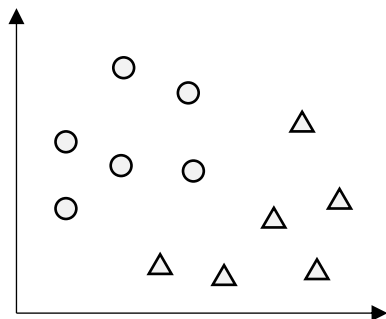
학습
→



보팅(voting)



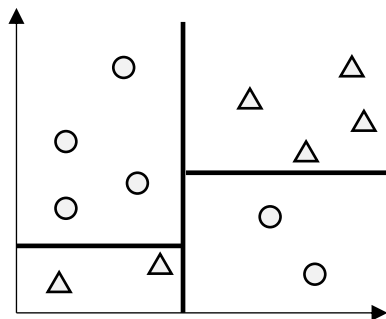
예측
→



예측
→



투표
→



예측
→



AI/BigData 인텐시브 과정

Section 1. 보팅

Section 1-2. 보팅 분류

보팅 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("./data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

보팅 분류

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2             'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

보팅 분류

```
In [4]: ▶ 1 # 트레이닝/테스트 데이터 분할
        2 from sklearn.model_selection import train_test_split
        3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [5]: ▶ 1 #데이터 표준화
        2 from sklearn.preprocessing import StandardScaler
        3 std_scale = StandardScaler()
        4 std_scale.fit(X_tn)
        5 X_tn_std = std_scale.transform(X_tn)
        6 X_te_std = std_scale.transform(X_te)
```

보팅 분류

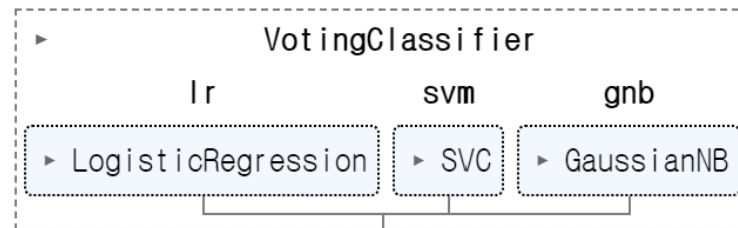
voting='hard' 기존 투표 방식
voting='soft' 클래스별 확률의 argmax

voting='soft'의 경우 개별 분류기 모두가
predict_proba 메소드를 지원해야 사용가능

In [6]: ▶

```
1 # 보팅 학습
2 from sklearn.linear_model import LogisticRegression
3 from sklearn import svm
4 from sklearn.naive_bayes import GaussianNB
5 from sklearn.ensemble import VotingClassifier
6
7 clf1 = LogisticRegression()
8 clf2 = svm.SVC(kernel='linear')
9 clf3 = GaussianNB()
10
11 clf_voting = VotingClassifier(
12     estimators=[
13         ('lr', clf1),
14         ('svm', clf2),
15         ('gnb', clf3)
16     ],
17     voting='hard',
18     weights=[1,1,1])
19 clf_voting.fit(X_tn_std, y_tn)
```

Out [6]:



보팅 분류

In [7]: ▶

```
1 # 예측
2 pred_voting = clf_voting.predict(X_te_std)
3 print(pred_voting)
```

```
[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 1 0 1 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```


보팅 분류

In [8]: ▶

```
1 # 정확도(accuracy)
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, pred_voting)
4 print(accuracy)
```

1.0

In [9]: ▶

```
1 # 정밀도(precision)
2 from sklearn.metrics import precision_score
3 precision = precision_score(y_te, pred_voting, average='macro')
4 print(precision)
```

1.0

In [10]: ▶

```
1 # 리콜
2 from sklearn.metrics import recall_score
3 recall = recall_score(y_te, pred_voting, average='macro')
4 print(recall)
```

1.0

In [11]: ▶

```
1 # f1 스코어
2 from sklearn.metrics import f1_score
3 f1 = f1_score(y_te, pred_voting, average='macro')
4 print(f1)
```

1.0

보팅 분류

In [12]: ▶

```
1 # confusion matrix 확인
2 from sklearn.metrics import confusion_matrix
3 conf_matrix = confusion_matrix(y_te, pred_voting)
4 print(conf_matrix)
```

```
[[16  0  0]
 [ 0 21  0]
 [ 0  0  8]]
```

In [13]: ▶

```
1 # 분류 레포트 확인
2 from sklearn.metrics import classification_report
3 class_report = classification_report(y_te, pred_voting)
4 print(class_report)
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
1	1.00	1.00	1.00	21
2	1.00	1.00	1.00	8
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

AI/BigData 인텐시브 과정

Section 1. 보팅

Section 1-3. 보팅 회귀

보팅 회귀

In [1]: ▶

1

import pandas as pd

2

df = pd.read_csv("../data/house_prices.csv")

In [2]: ▶

1

df

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

보팅 회귀

```
In [3]: ▶ 1 df['MEDV'].unique()
```

```
Out[3]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
               21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
               15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
               24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
               25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
               17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
               22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
               20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
               14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
               15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
               19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
               32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
               42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
               31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
               44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
               25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
               28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
               10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
               12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
               7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
               10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

보팅회귀

In [4]: ▶

1 df.columns

Out[4]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV'], dtype='object')

In [5]: ▶

1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
2
3 X = df[features]
4 y = df['MEDV']

In [6]: ▶

1 X

Out[6]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 13 columns

보팅 회귀

In [7]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [8]: ▶

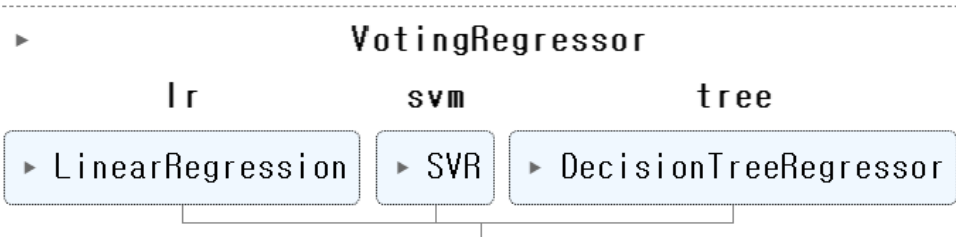
```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

보팅 회귀

In [11]: ▶

```
1 # 보팅 학습
2 from sklearn.linear_model import LinearRegression
3 from sklearn.svm import SVR
4 from sklearn.tree import DecisionTreeRegressor
5 from sklearn.ensemble import VotingRegressor
6
7 clf1 = LinearRegression()
8 clf2 = SVR(kernel='linear')
9 clf3 = DecisionTreeRegressor()
10
11 clf_voting = VotingRegressor(
12     estimators=[
13         ('lr', clf1),
14         ('svm', clf2),
15         ('tree', clf3)
16     ],
17     weights=[1,1,1])
18 clf_voting.fit(X_tn_std, y_tn)
```

Out[11]:



보팅 회귀

[23]:

```
# 예측
vote_pred = clf_voting.predict(X_te_std)
print(vote_pred)
```

```
[24.39393609 22.99041152 26.26088334 12.66693241 21.45715018 19.57609632
 19.88289154 20.39116962 16.8822151 18.16406136 5.75442668 16.25872001
 15.45084565 5.30934232 42.44536865 33.79781264 21.38009918 35.64294775
 27.6904101 21.81182962 24.62101585 22.68379461 20.19961558 27.99951063
 21.9744006 7.9079855 17.52940295 16.97498899 36.31589779 19.6229526
 15.52269468 17.47430599 21.18448971 22.79528905 25.84538555 19.69576622
 10.71570332 22.47555318 14.83797106 15.90075634 25.01158792 21.89525482
 22.03438084 13.21430983 22.69739201 23.41568481 20.42871108 19.42664087
 11.13606009 25.35703465 17.92048722 18.40222839 23.57110782 33.03694727
 13.49079694 20.55407087 20.19174533 16.34043045 10.88392026 21.5929576
 18.87755429 22.00922522 31.50026085 27.85874677 18.85474541 29.1951345
 18.62933839 19.41705298 15.68023948 22.8114743 21.74439156 23.09648418
 27.81836487 29.24039784 25.39527894 5.60446412 38.96963767 23.42477182
 25.03271824 20.79218456 27.67411143 17.85054035 18.26721927 37.71843947
 42.63584517 24.21589361 22.9402144 14.15124703 25.19375095 16.31614531
 16.36222698 12.82027352 22.96600971 30.24175912 21.5367938 21.44401006
 3.11066382 24.43105733 14.4189909 17.82551028 25.14875258 21.42235008
 30.97740604 22.82622806 25.23553043 21.44979803 6.05063581 13.83726523
 22.6413182 26.38768203 33.26213449 14.08831003 19.47046575 19.86516269
 14.66399937 22.20563545 6.18358067 19.65922999 9.08549686 45.91226899
 30.02583262 10.03650007 16.28804265 21.1517584 23.48063161 20.0533002
 35.00334527]
```

[24]:

```
# 모형 평가-MSE
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_te, vote_pred)
print(mse)
```

```
26.803961946215832
```

AI/BigData 인텐시브 과정

Section 2. 배경

Section 2-1. 배경 개념

배깅(bagging)

- 개별 분류기들의 분류 결과를 종합 후 최종 분류기 성능 향상
- 부트스트랩(bootstrap) 샘플을 뽑아 학습
- 부트스트랩(bootstrap)이란 중복을 허용한 랜덤 샘플 방법을 의미

랜덤 포레스트(random forest)

Decision Tree

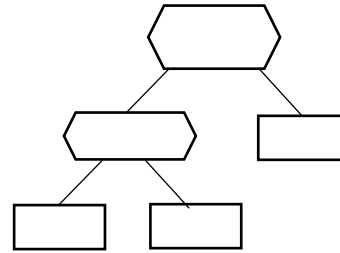
나무

Random Forest

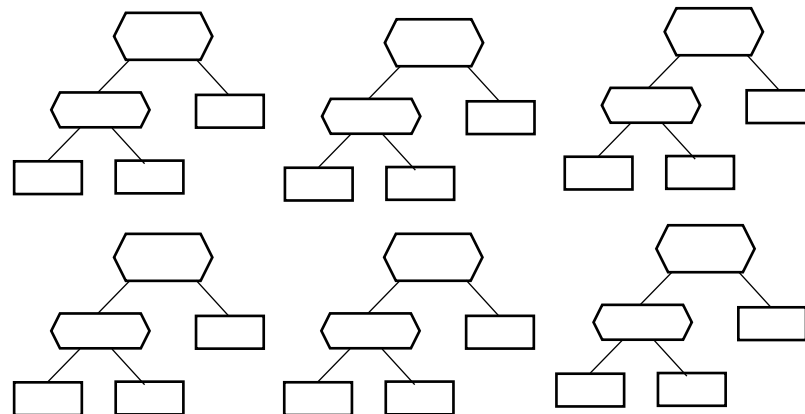
숲

랜덤 포레스트(random forest)

Decision Tree



Random Forest



랜덤 포레스트(random forest)

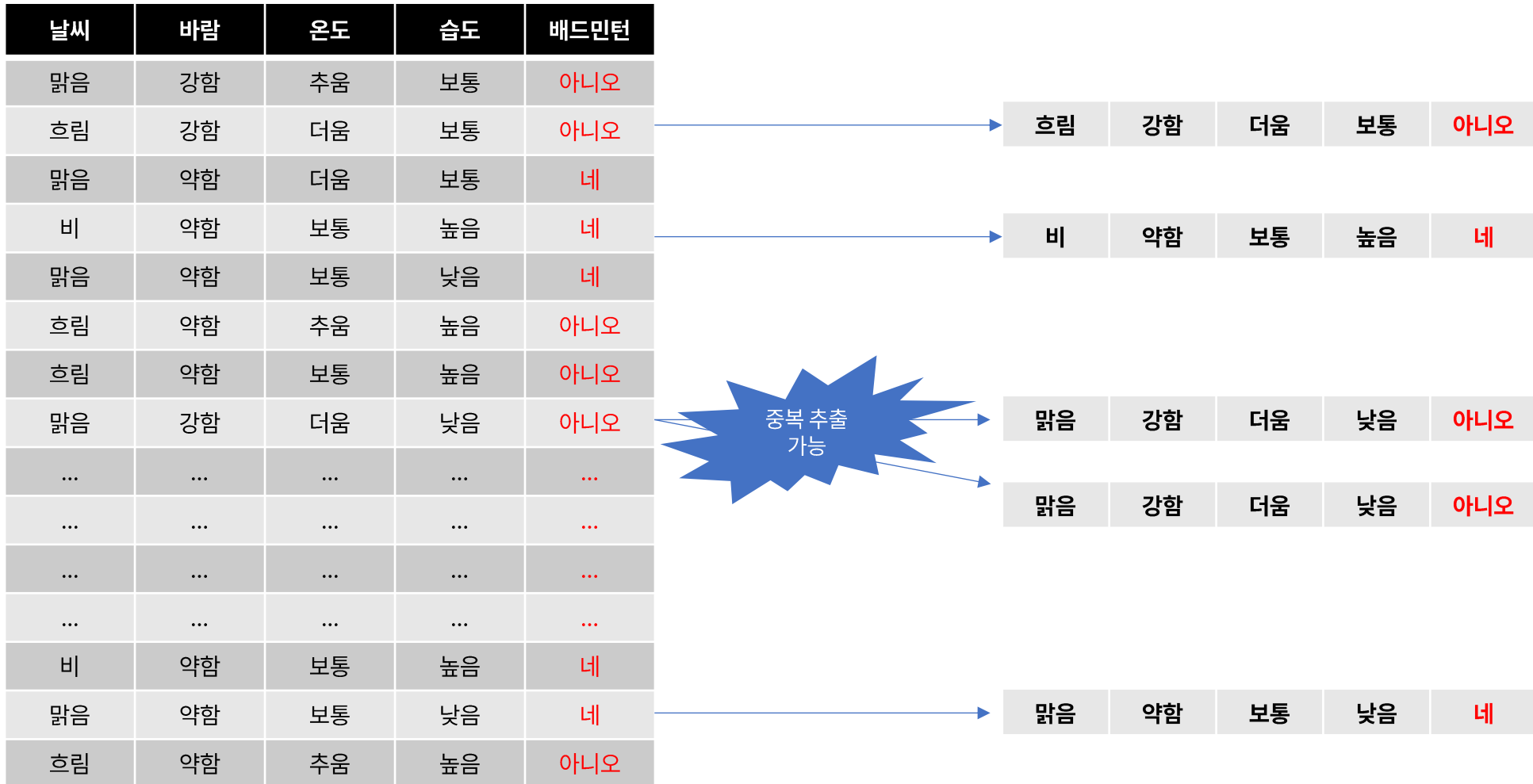
n개의 랜덤 데이터 샘플 선택(중복 가능)

d개의 피쳐 선택(중복 불가능)

Decision Tree 학습

각 Decision Tree 결과의 투표를 통해 클래스 할당

랜덤 포레스트(random forest)



랜덤 포레스트(random forest)

➤ 부트스트랩: 중복을 허용한 랜덤 샘플링

트레이닝 데이터에서 중복을 허용하여 랜덤 샘플링



배깅(bagging)

bootstrap aggregating

랜덤 포레스트(random forest)

n개의 랜덤 데이터 샘플 선택(중복 가능)

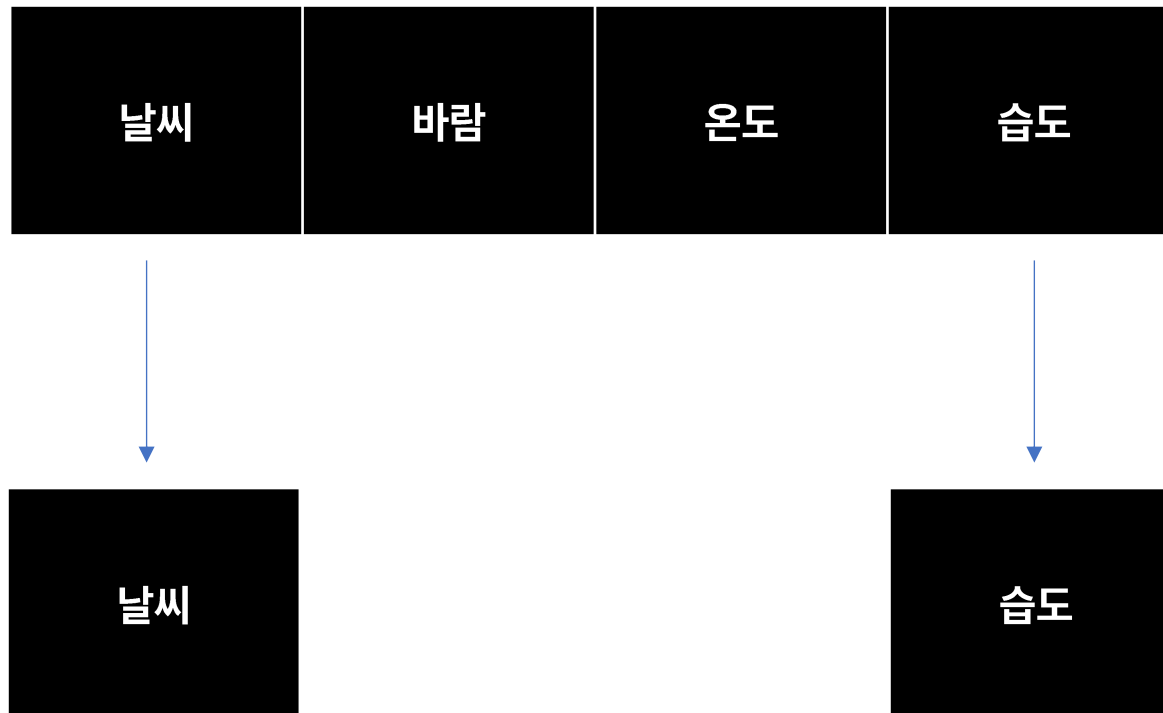
d개의 피쳐 선택(중복 불가능)

Decision Tree 학습

각 Decision Tree 결과의 투표를 통해 클래스 할당

랜덤 포레스트(random forest)

➤ d개의 피쳐 선택: 피쳐 선택시에는 중복 허용 안됨



랜덤 포레스트(random forest)

n개의 랜덤 데이터 샘플 선택(중복 가능)

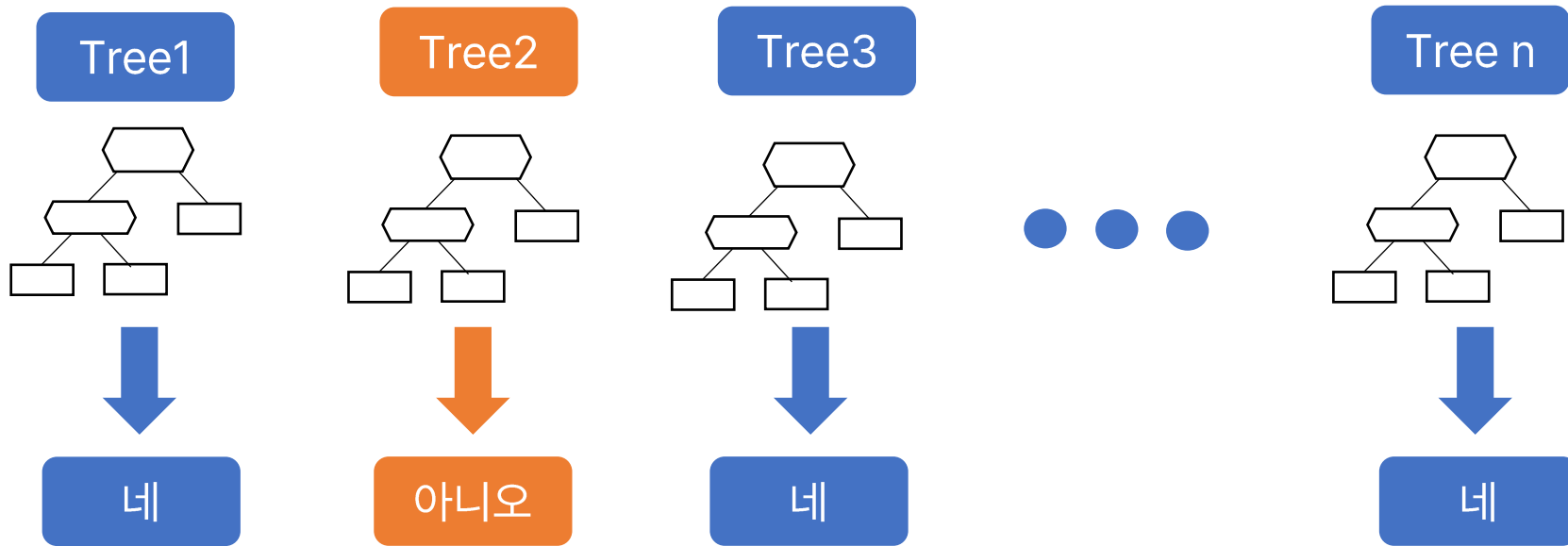
d개의 피쳐 선택(중복 불가능)

Decision Tree 학습

각 Decision Tree 결과의 투표를 통해 클래스 할당

랜덤 포레스트(random forest)

➤ d개의 피쳐 선택: 피쳐 선택시에는 중복 허용 안됨



랜덤 포레스트(random forest)

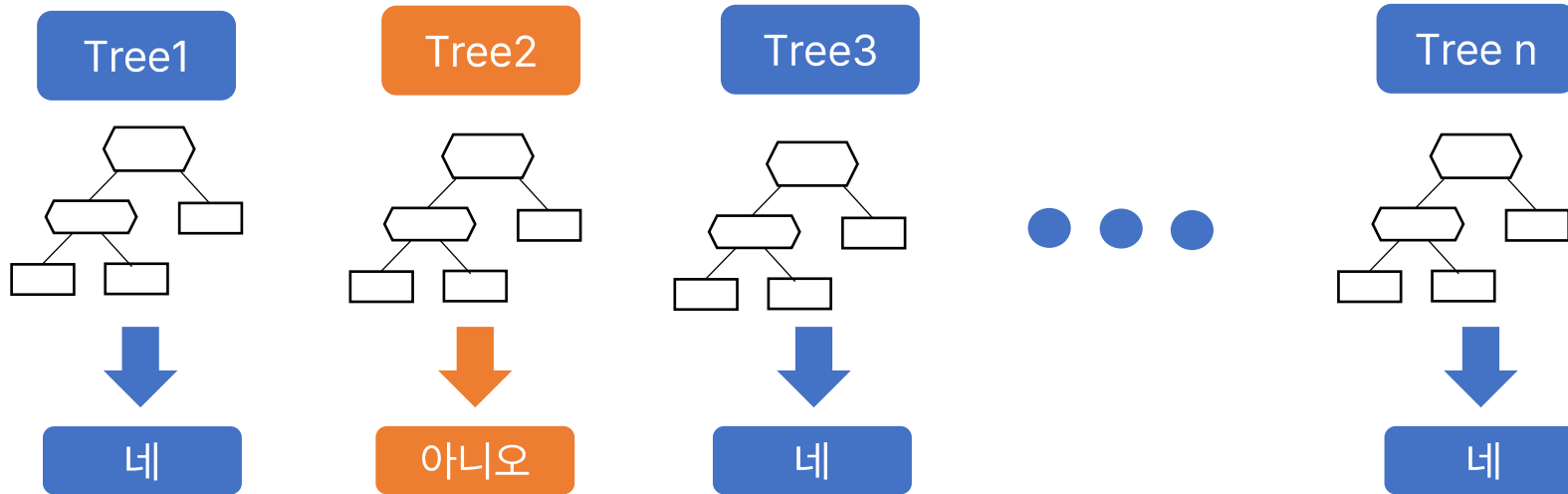
n개의 랜덤 데이터 샘플 선택(중복 가능)

d개의 피쳐 선택(중복 불가능)

Decision Tree 학습

각 Decision Tree 결과의 투표를 통해 클래스 할당

랜덤 포레스트(random forest)



날씨	바람	온도	습도	Tree1 결과	Tree2 결과	Tree3 결과	...	Tree n 결과
맑음	강함	추움	보통	네	아니오	네	...	네

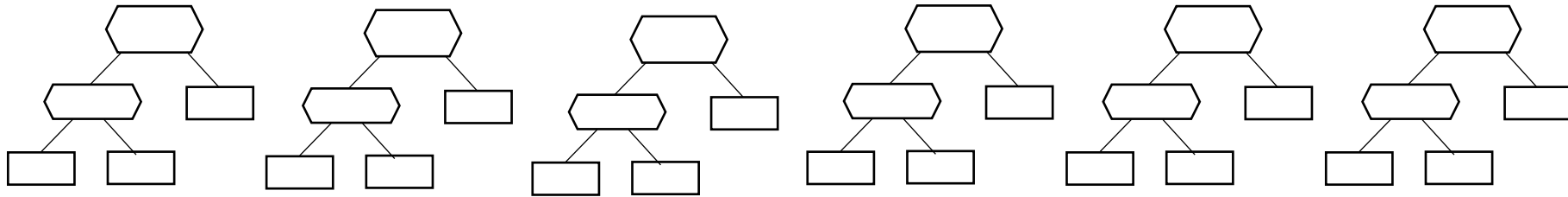
랜덤 포레스트(random forest)

날씨	바람	온도	습도	Tree1 결과	Tree2 결과	Tree3 결과	...	Tree n 결과
맑음	강함	추움	보통	네	아니오	네	...	네

네 아니오
 VS
7표 3표

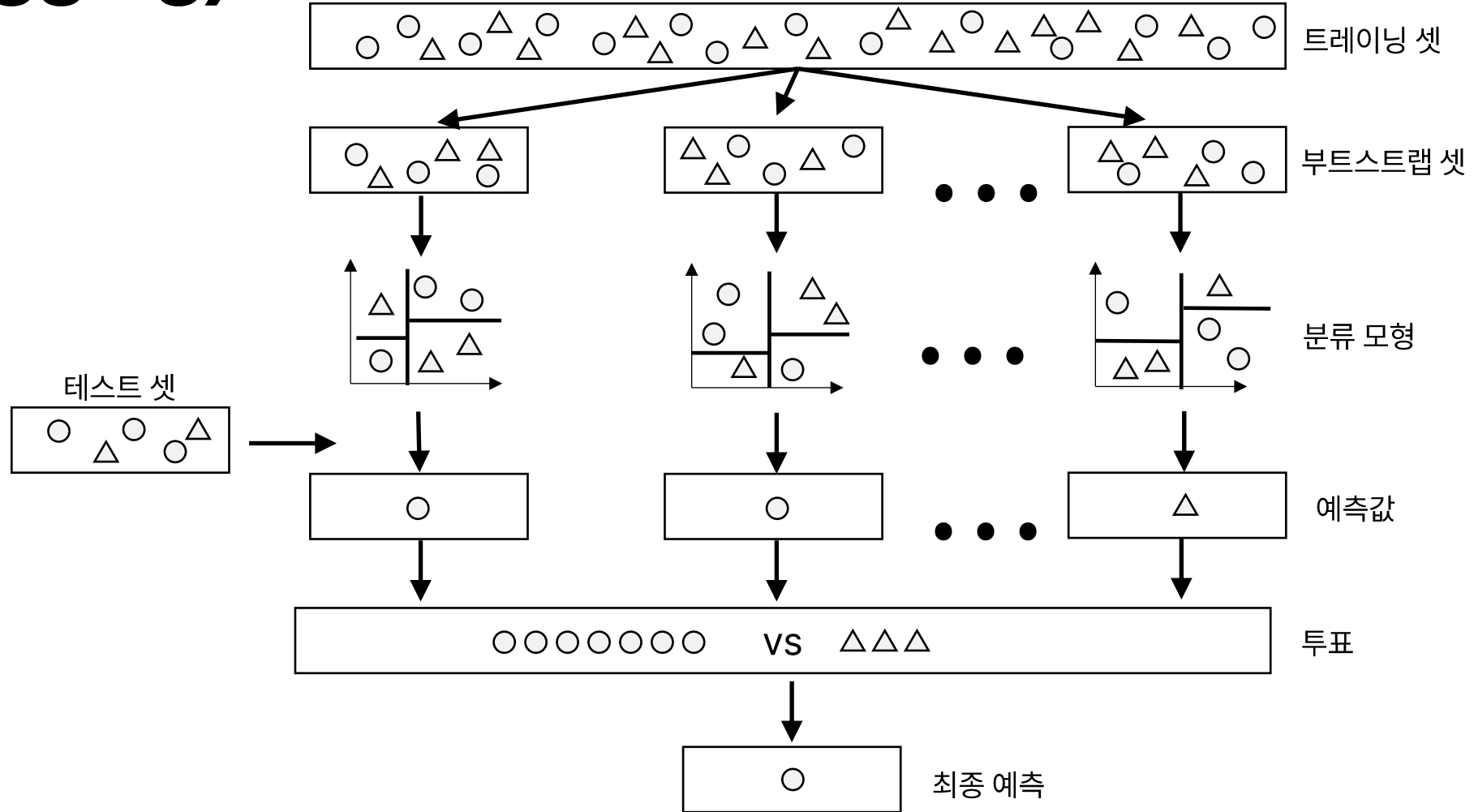
날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	네

랜덤 포레스트(random forest)



각 트리별 의견 종합

배깅(bagging)



AI/BigData 인텐시브 과정

Section 2. 배경

Section 2-2. 랜덤 포레스트 분류

랜덤 포레스트 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("./data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

랜덤 포레스트 분류

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

랜덤 포레스트 분류

In [4]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

랜덤 포레스트 분류

```
In [6]: ▶ 1 from sklearn.ensemble import RandomForestClassifier
          2 clf_rf = RandomForestClassifier(max_depth=2,
          3                                   random_state=0)
          4 clf_rf.fit(X_tn_std, y_tn)
```

```
Out[6]: ▼      RandomForestClassifier
RandomForestClassifier(max_depth=2, random_state=0)
```

```
In [7]: ▶ 1 # 예측
          2 pred_rf = clf_rf.predict(X_te_std)
          3 print(pred_rf)

[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 2 0 0 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```

랜덤 포레스트 분류

In [8]: ▶

```
1 # 정확도(accuracy)
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, pred_rf)
4 print(accuracy)
```

0.9555555555555556

In [9]: ▶

```
1 # 정밀도(precision)
2 from sklearn.metrics import precision_score
3 precision = precision_score(y_te, pred_rf, average='macro')
4 print(precision)
```

0.943355119825708

In [10]: ▶

```
1 # 리콜
2 from sklearn.metrics import recall_score
3 recall = recall_score(y_te, pred_rf, average='macro')
4 print(recall)
```

0.9682539682539683

In [11]: ▶

```
1 # f1 스코어
2 from sklearn.metrics import f1_score
3 f1 = f1_score(y_te, pred_rf, average='macro')
4 print(f1)
```

0.9536244800950685

랜덤 포레스트 분류

```
In [12]: ▶ 1 # confusion matrix 확인
          2 from sklearn.metrics import confusion_matrix
          3 conf_matrix = confusion_matrix(y_te, pred_rf)
          4 print(conf_matrix)
```

```
[[16  0  0]
 [ 1 19  1]
 [ 0  0  8]]
```

```
In [13]: ▶ 1 # 분류 레포트 확인
          2 from sklearn.metrics import classification_report
          3 class_report = classification_report(y_te, pred_rf)
          4 print(class_report)
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	16
1	1.00	0.90	0.95	21
2	0.89	1.00	0.94	8
accuracy			0.96	45
macro avg	0.94	0.97	0.95	45
weighted avg	0.96	0.96	0.96	45

AI/BigData 인텐시브 과정

Section 2. 배경

Section 2-3. 랜덤 포레스트 회귀

랜덤 포레스트 회귀

In [1]: ▶

1

import pandas as pd

2

df = pd.read_csv("./data/house_prices.csv")

In [2]: ▶

1

df

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

랜덤 포레스트 회귀

```
In [3]: ▶ 1 df['MEDV'].unique()
```

```
Out[3]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
                21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
                15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
                24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
                25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
                17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
                22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
                20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
                14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
                15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
                19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
                32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
                42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
                31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
                44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
                25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
                28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
                10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
                12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
                7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
                10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

랜덤포레스트회귀

In [4]: ▶

1 df.columns

Out[4]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV'], dtype='object')

In [5]: ▶

1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
2
3 X = df[features]
4 y = df['MEDV']

In [6]: ▶

1 X

Out[6]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 13 columns

랜덤 포레스트 회귀

In [7]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [8]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

Section

랜덤포레스트회귀

```
In [9]: 1 from sklearn.ensemble import RandomForestRegressor
2 clf_rf = RandomForestRegressor(max_depth=2,
3                               random_state=0)
4 clf_rf.fit(X_tn_std, y_tn)
```

```
Out[9]: RandomForestRegressor
RandomForestRegressor(max_depth=2, random_state=0)
```

```
In [12]: 1 # 예측
2 rf_pred = clf_rf.predict(X_te_std)
3 print(rf_pred)

[25.13026738 22.45503023 21.72597661 14.02746048 21.72597661 23.39001805
 21.72597661 23.25907479 21.72597661 21.72597661 14.02746048 14.64220662
 15.06631268 14.02746048 43.87712974 31.67863592 21.65508147 32.81235452
 25.29273146 23.32331968 25.13026738 21.72597661 21.72597661 25.13026738
 21.72597661 20.30936428 21.72597661 15.44912052 43.05952486 15.44407281
 14.67676803 17.77333379 21.78263112 21.72597661 23.25907479 17.82865762
 14.02746048 22.58953053 14.47304952 15.1343252 23.39001805 21.72597661
 23.25907479 14.67676803 23.46698736 23.25907479 16.09842807 15.6439473
 14.79476526 23.65611834 16.17750951 15.44407281 21.72597661 43.73480054
 15.44407281 16.09842807 21.72597661 21.72597661 20.36601879 15.44407281
 24.50827659 21.72597661 30.60709929 26.49005792 17.54830779 25.79420234
 16.05881895 15.44407281 19.89513254 21.72597661 21.72597661 23.32331968
 25.64042788 28.97548892 24.90532013 14.02746048 43.44101044 21.72597661
 23.32331968 21.72597661 25.43803727 17.82865762 20.30936428 43.05952486
 42.81452885 25.13026738 21.72597661 15.12740478 25.13026738 15.44407281
 18.31561257 15.29151416 25.13026738 28.46532842 23.97120964 23.25907479
 14.02746048 25.13026738 15.44407281 21.72597661 23.25907479 21.72597661
 25.86887978 22.13241693 24.81381208 21.72597661 14.02746048 21.07666906
 23.25907479 23.39001805 29.26856252 14.02746048 15.44407281 16.16543356
 15.44407281 18.47796517 14.02746048 21.72597661 14.02746048 43.73480054
 26.17980062 14.02746048 15.7325048 21.72597661 21.78263112 15.2403543
 32.51335106]
```

```
In [13]: 1 # 모형 평가-MSE
2 from sklearn.metrics import mean_squared_error
3 mse = mean_squared_error(y_te, rf_pred)
4 print(mse)
```

27.07190389377867

AI/BigData 인텐시브 과정

Section 2. 배경

Section 2-3. 배경 분류

배경 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

배경 분류

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

배경 분류

```
In [4]: ▶ 1 # 트레이닝/테스트 데이터 분할
          2 from sklearn.model_selection import train_test_split
          3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [5]: ▶ 1 #데이터 표준화
          2 from sklearn.preprocessing import StandardScaler
          3 std_scale = StandardScaler()
          4 std_scale.fit(X_tn)
          5 X_tn_std = std_scale.transform(X_tn)
          6 X_te_std = std_scale.transform(X_te)
```

배깅 분류

In [14]: ▶

```
1 # 배깅 학습
2 from sklearn.naive_bayes import GaussianNB
3 from sklearn.ensemble import BaggingClassifier
4 clf_bagging = BaggingClassifier(estimator=GaussianNB(),
5                                n_estimators=10,
6                                random_state=0)
7 clf_bagging.fit(X_tn_std, y_tn)
```

Out[14]:

```
▶ BaggingClassifier
▶ estimator: GaussianNB
  ▶ GaussianNB
```

In [7]: ▶

```
1 # 예측
2 pred_bagging = clf_bagging.predict(X_te_std)
3 print(pred_bagging)
```

```
[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 2 0 0 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```

배깅 분류

```
In [8]: ▶ 1 # 정확도(accuracy)
          2 from sklearn.metrics import accuracy_score
          3 accuracy = accuracy_score(y_te, pred_bagging)
          4 print(accuracy)
```

0.9555555555555556

```
In [9]: ▶ 1 # 정밀도(precision)
          2 from sklearn.metrics import precision_score
          3 precision = precision_score(y_te, pred_bagging, average='macro')
          4 print(precision)
```

0.943355119825708

```
In [10]: ▶ 1 # 리콜
           2 from sklearn.metrics import recall_score
           3 recall = recall_score(y_te, pred_bagging, average='macro')
           4 print(recall)
```

0.9682539682539683

```
In [11]: ▶ 1 # f1 score
           2 from sklearn.metrics import f1_score
           3 f1 = f1_score(y_te, pred_bagging, average='macro')
           4 print(f1)
```

0.9536244800950685

배깅 분류

```
In [12]: ▶ 1 # confusion matrix 확인
          2 from sklearn.metrics import confusion_matrix
          3 conf_matrix = confusion_matrix(y_te, pred_bagging)
          4 print(conf_matrix)
```

```
[[16  0  0]
 [ 1 19  1]
 [ 0  0  8]]
```

```
In [13]: ▶ 1 # 분류 레포트 확인
          2 from sklearn.metrics import classification_report
          3 class_report = classification_report(y_te, pred_bagging)
          4 print(class_report)
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	16
1	1.00	0.90	0.95	21
2	0.89	1.00	0.94	8
accuracy			0.96	45
macro avg	0.94	0.97	0.95	45
weighted avg	0.96	0.96	0.96	45

AI/BigData 인텐시브 과정

Section 2. 배경

Section 2-3. 배경 회귀

배깅 회귀

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("./data/house_prices.csv")
```

```
In [2]: 1 df
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

배깅 회귀

```
In [3]: ▶ 1 df['MEDV'].unique()
```

```
Out[3]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,  
                21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,  
                15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,  
                24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,  
                25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,  
                17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,  
                22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,  
                20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,  
                14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,  
                15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,  
                19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,  
                32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,  
                42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,  
                31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,  
                44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,  
                25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,  
                28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,  
                10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,  
                12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,  
                7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,  
                10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```


Section 배경화귀

In [4]: 1 df.columns

Out[4]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
'PTRATIO', 'B', 'LSTAT', 'MEDV'],
dtype='object')

In [5]: 1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
2
3 X = df[features]
4 y = df['MEDV']

In [6]: 1 X

Out[6]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows x 13 columns

배깅 회귀

In [7]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [8]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

배깅 회귀

```
In [10]: ▶ 1 # 배깅 학습
2 from sklearn.svm import SVR
3 from sklearn.ensemble import BaggingRegressor
4 clf_bagging = BaggingRegressor(estimator=SVR(),
5                               n_estimators=10,
6                               random_state=0)
7 clf_bagging.fit(X_tn_std, y_tn)
```

Out[10]:

```
▶ BaggingRegressor
  ▶ estimator: SVR
    ▶ SVR
```

```
In [11]: ▶ 1 # 예측
2 bagging_pred = clf_bagging.predict(X_te_std)
3 print(bagging_pred)

[25.2522426  18.87848192 24.35009668 13.28158615 21.13493963 19.39724273
 21.07311554 20.18464981 18.85140479 18.843497  21.01057126 13.26291355
 14.60210982 12.82664591 26.60853822 29.55363494 21.74705357 30.64977646
 29.06843519 21.56444462 24.01279831 21.46296621 19.0277086  27.97537236
 20.51498561 17.82764972 17.1241731  18.38243736 33.82635379 18.80485556
 15.24989978 17.3281799  19.65085015 21.3890741  26.09714587 20.65899624
 12.13203952 22.03170547 13.87085127 13.10019667 23.54346299 20.31505234]
```

```
In [12]: ▶ 1 # 모형 평가-MSE
2 from sklearn.metrics import mean_squared_error
3 mse = mean_squared_error(y_te, bagging_pred)
4 print(mse)

40.730001386173015
```

AI/BigData 인텐시브 과정

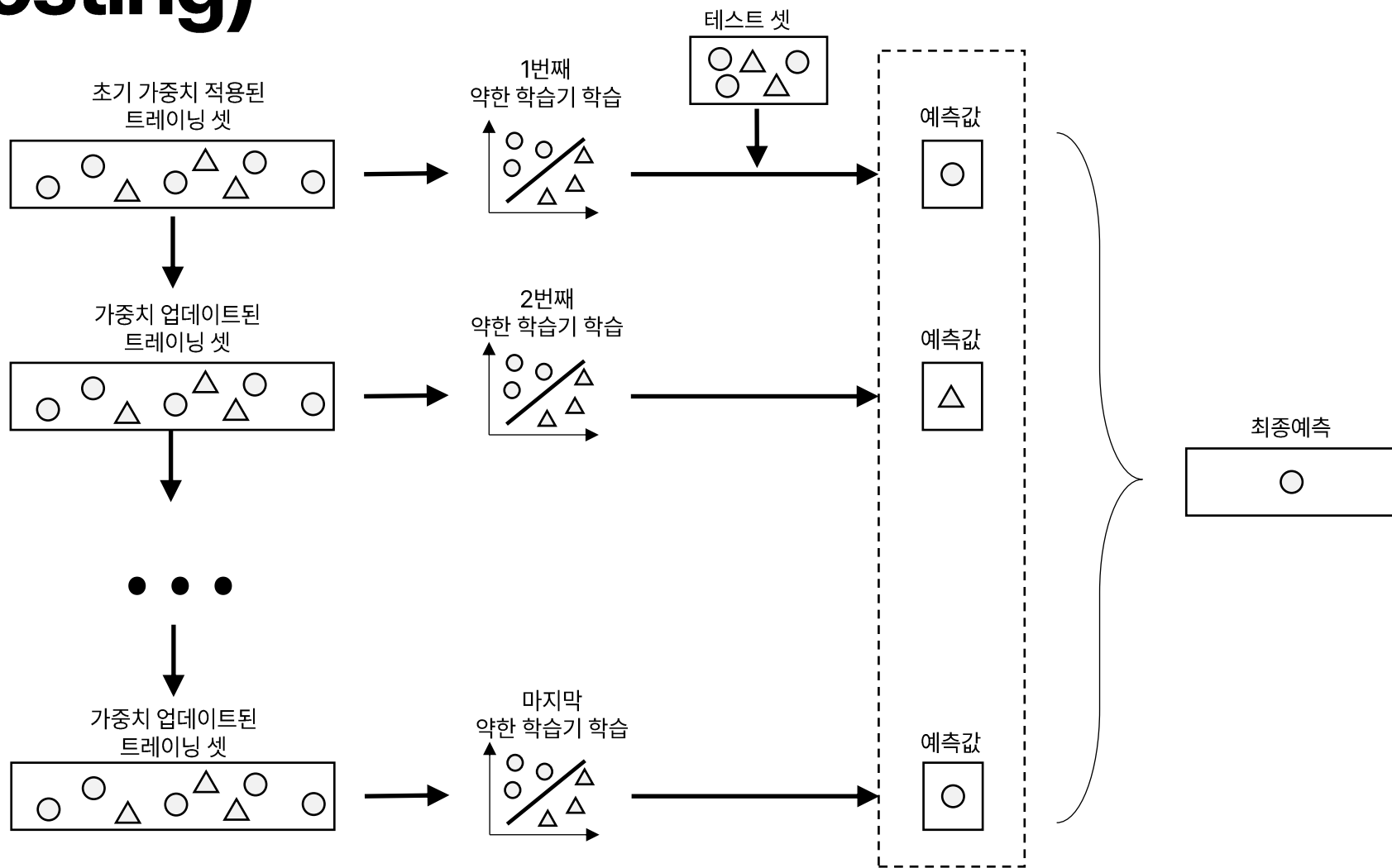
Section 3. 부스팅

Section 3-1. 부스팅 개념

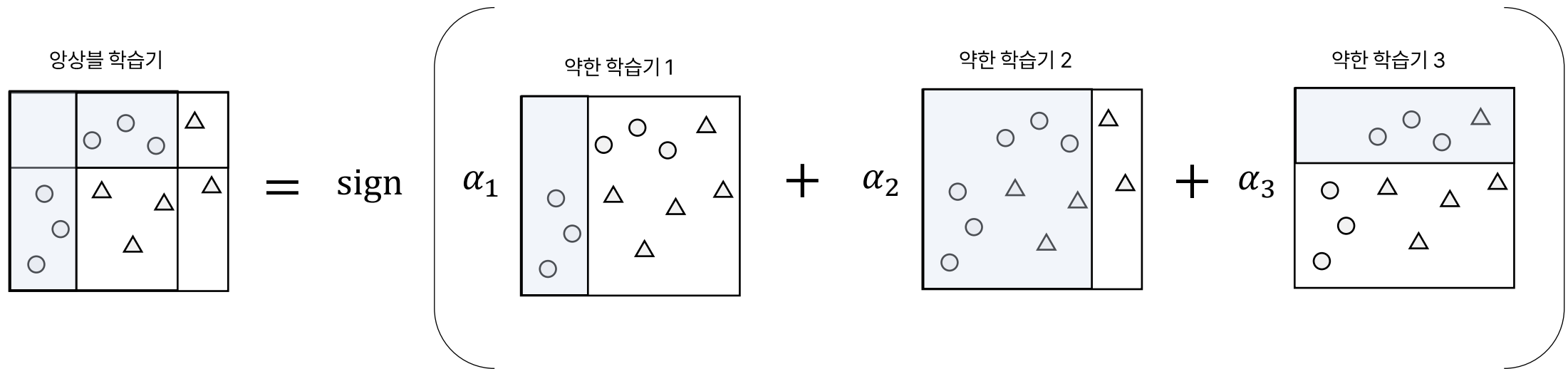
부스팅(boosting)

- 분류하기 어려운 데이터 포인트에 가중치를 할당
- 학습이 진행되면서 올바르게 분류된 데이터의 가중치는 감소
- 학습이 진행되면서 잘못 분류된 데이터의 가중치는 증가
- 학습이 진행되면서 학습기는 분류하기 어려운 데이터에 집중하게 됨

부스팅(boosting)



부스팅(boosting)



AI/BigData 인텐시브 과정

Section 3. 부스팅

Section 3-2. 에이다 부스트 분류

에이다 부스트(ada boost) 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("./data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

에이다 부스트(ada boost) 분류

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

에이다 부스트(ada boost) 분류

In [4]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

에이다 부스트(ada boost) 분류

In [6]: ▶

```
1 # 에이다 부스트 학습
2 from sklearn.ensemble import AdaBoostClassifier
3 clf_ada = AdaBoostClassifier(random_state=0)
4 clf_ada.fit(X_tn_std, y_tn)
```

Out[6]:

```
▼ AdaBoostClassifier
AdaBoostClassifier(random_state=0)
```

In [7]: ▶

```
1 # 예측
2 pred_ada = clf_ada.predict(X_te_std)
3 print(pred_ada)
```

```
[0 2 0 0 1 0 0 2 1 1 2 2 0 0 2 1 0 0 1 0 0 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 0 2 1 0 1 1 1]
```

에이다 부스트(ada boc

In [8]: ▶

```
1 # 정확도(accuracy)
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, pred_ada)
4 print(accuracy)
```

0.8666666666666667

In [9]: ▶

```
1 # 정밀도(precision)
2 from sklearn.metrics import precision_score
3 precision = precision_score(y_te, pred_ada, average='macro')
4 print(precision)
```

0.8970588235294118

In [10]: ▶

```
1 # 리콜
2 from sklearn.metrics import recall_score
3 recall = recall_score(y_te, pred_ada, average='macro')
4 print(recall)
```

0.8998015873015873

In [11]: ▶

```
1 # f1 스코어
2 from sklearn.metrics import f1_score
3 f1 = f1_score(y_te, pred_ada, average='macro')
4 print(f1)
```

0.8918128654970761

에이다 부스트(ada boost) 분류

In [12]: ▶

```
1 # confusion matrix 확인
2 from sklearn.metrics import confusion_matrix
3 conf_matrix = confusion_matrix(y_te, pred_ada)
4 print(conf_matrix)
```

```
[[15  1  0]
 [ 5 16  0]
 [ 0  0  8]]
```

In [13]: ▶

```
1 # 분류 레포트 확인
2 from sklearn.metrics import classification_report
3 class_report = classification_report(y_te, pred_ada)
4 print(class_report)
```

	precision	recall	f1-score	support
0	0.75	0.94	0.83	16
1	0.94	0.76	0.84	21
2	1.00	1.00	1.00	8
accuracy			0.87	45
macro avg	0.90	0.90	0.89	45
weighted avg	0.88	0.87	0.87	45

AI/BigData 인텐시브 과정

Section 3. 부스팅

Section 3-3. 에이다부스트 회귀

에이다 부스트(ada boost) 회귀

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/house_prices.csv")
```

```
In [2]: 1 df
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

에이다 부스트(ada boost) 회귀

```
In [3]: ▶ 1 df['MEDV'].unique()
```

```
Out[3]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
                21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
                15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
                24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
                25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
                17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
                22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
                20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
                14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
                15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
                19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
                32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
                42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
                31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
                44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
                25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
                28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
                10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
                12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
                7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
                10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

에이다부스트회귀

```
In [4]: 1 df.columns
```

```
Out[4]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',  
              'PTRATIO', 'B', 'LSTAT', 'MEDV'],  
              dtype='object')
```

```
In [5]: 1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']  
2  
3 X = df[features]  
4 y = df['MEDV']
```

```
In [6]: 1 X
```

```
Out[6]:
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 13 columns

에이다 부스트(ada boost) 회귀

```
In [7]: ▶ 1 # 트레이닝/테스트 데이터 분할  
2 from sklearn.model_selection import train_test_split  
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [8]: ▶ 1 #데이터 표준화  
2 from sklearn.preprocessing import StandardScaler  
3 std_scale = StandardScaler()  
4 std_scale.fit(X_tn)  
5 X_tn_std = std_scale.transform(X_tn)  
6 X_te_std = std_scale.transform(X_te)
```

에이다 부스트(ada boost) 회귀

```
In [10]: ▶ 1 # 에이다 부스트 학습
          2 from sklearn.ensemble import AdaBoostRegressor
          3 clf_ada = AdaBoostRegressor(random_state=0)
          4 clf_ada.fit(X_tn_std, y_tn)
```

```
Out[10]: ▼ AdaBoostRegressor
          AdaBoostRegressor(random_state=0)
```

```
In [11]: ▶ 1 # 예측
          2 ada_pred = clf_ada.predict(X_te_std)
          3 print(ada_pred)

[26.35      23.11636364 23.52916667 11.89384615 19.80540541 23.69934211
 21.4972973 24.38522727 19.80540541 22.19166667 10.646875 12.85
 15.32564103 10.78035714 46.03448276 32.96235294 21.4972973 33.45555556
 26.35      23.69934211 24.66363636 19.57037037 19.14354839 27.02346939
 22.75      15.77391304 19.11111111 15.77391304 42.8      18.26162791
 15.45625   17.97017544 22.46703297 20.06266667 24.925625 15.77391304]
```

```
In [12]: ▶ 1 # 모형 평가-MSE
          2 from sklearn.metrics import mean_squared_error
          3 mse = mean_squared_error(y_te, ada_pred)
          4 print(mse)

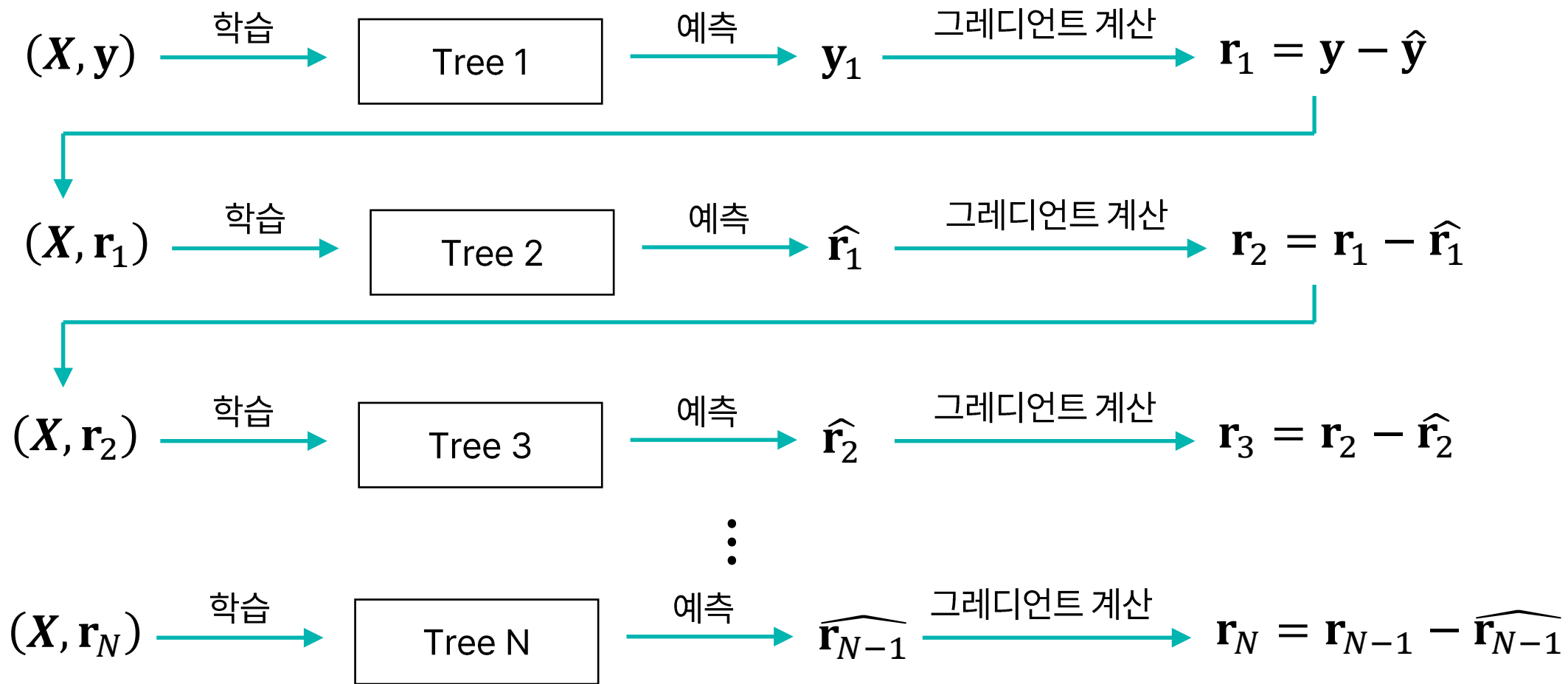
25.17393732173523
```

AI/BigData 인텐시브 과정

Section 3. 부스팅

Section 3-4. 그래디언트 부스트 분류

$$\hat{y} = y_1 + (\eta \times \mathbf{r}_1) + (\eta \times \mathbf{r}_2) + \cdots + (\eta \times \mathbf{r}_N)$$



그래디언트 부스팅(gradient boosting) 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

그래디언트 부스팅(gradient boosting) 분류

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```


그래디언트 부스팅(gradient boosting) 분류

In [4]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

그래디언트 부스팅(gradient boosting) 분류

In [6]: ▶

```
1 # Gradient Boosting 학습
2 from sklearn.ensemble import GradientBoostingClassifier
3 clf_gbt = GradientBoostingClassifier(max_depth=2,
4                                     learning_rate=0.1,
5                                     random_state=0)
6 clf_gbt.fit(X_tn_std, y_tn)
```

Out[6]:

```
▼ GradientBoostingClassifier
GradientBoostingClassifier(max_depth=2, random_state=0)
```

In [7]: ▶

```
1 # 예측
2 pred_gboost = clf_gbt.predict(X_te_std)
3 print(pred_gboost)
```

```
[0 2 1 0 1 0 0 2 1 1 2 2 0 1 2 1 0 0 2 0 1 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```

그래디언트 부스팅(gb)

```
In [8]: ▶ 1 # 정확도(accuracy)
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, pred_gboost)
4 print(accuracy)
```

0.9555555555555556

```
In [9]: ▶ 1 # 정밀도(precision)
2 from sklearn.metrics import precision_score
3 precision = precision_score(y_te, pred_gboost, average='macro')
4 print(precision)
```

0.943355119825708

```
In [10]: ▶ 1 # 리콜
2 from sklearn.metrics import recall_score
3 recall = recall_score(y_te, pred_gboost, average='macro')
4 print(recall)
```

0.9682539682539683

```
In [11]: ▶ 1 # f1 스코어
2 from sklearn.metrics import f1_score
3 f1 = f1_score(y_te, pred_gboost, average='macro')
4 print(f1)
```

0.9536244800950685

그래디언트 부스팅(gradient boosting) 분류

In [12]: ▶

```
1 # confusion matrix 확인
2 from sklearn.metrics import confusion_matrix
3 conf_matrix = confusion_matrix(y_te, pred_gboost)
4 print(conf_matrix)
```

```
[[16  0  0]
 [ 1 19  1]
 [ 0  0  8]]
```

In [13]: ▶

```
1 # 분류 레포트 확인
2 from sklearn.metrics import classification_report
3 class_report = classification_report(y_te, pred_gboost)
4 print(class_report)
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	16
1	1.00	0.90	0.95	21
2	0.89	1.00	0.94	8
accuracy			0.96	45
macro avg	0.94	0.97	0.95	45
weighted avg	0.96	0.96	0.96	45

AI/BigData 인텐시브 과정

Section 3. 부스팅

Section 3-4. 그래디언트 부스트 회귀

그래디언트 부스팅(gradient boosting) 회귀

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/house_prices.csv")
```

```
In [2]: 1 df
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

그래디언트 부스팅(gradient boosting) 회귀

```
In [3]: 1 df['MEDV'].unique()
```

```
Out[3]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
               21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
               15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
               24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
               25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
               17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
               22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
               20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
               14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
               15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
               19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
               32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
               42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
               31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
               44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
               25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
               28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
               10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
               12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
               7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
               10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

그래디언트부스트회귀

In [4]:

1

df.columns

Out[4]:

Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
 'PTRATIO', 'B', 'LSTAT', 'MEDV'],
 dtype='object')

In [5]:

1

features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']

2

3

4

X = df[features]

y = df['MEDV']

In [6]:

1

X

Out[6]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 13 columns

그래디언트 부스팅(gradient boosting) 회귀

In [7]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [8]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

그래디언트 부스팅(gradient boosting) 회귀

```
In [9]: ▶ 1 # Gradient Boosting 학습
        2 from sklearn.ensemble import GradientBoostingRegressor
        3 clf_gbt = GradientBoostingRegressor(max_depth=2,
        4                                     learning_rate=0.1,
        5                                     random_state=0)
        6 clf_gbt.fit(X_tn_std, y_tn)
```

```
Out[9]: ▾ GradientBoostingRegressor
        GradientBoostingRegressor(max_depth=2, random_state=0)
```

```
In [12]: ▶ 1 # 예측
        2 gbt_pred = clf_gbt.predict(X_te_std)
        3 print(gbt_pred)

[24.95902493 22.01220541 23.63627037 11.07373229 21.62505687 20.00119094
 20.41069776 20.90294929 22.06702837 19.2749568   8.35656579 15.68726577
 14.8727825  10.65710962 48.96936901 35.08601386 21.09745751 36.60967375
 25.74582497 21.73028861 23.14480637 21.700729   20.55494592 24.59037383]
```

```
In [13]: ▶ 1 # 모형 평가-MSE
        2 from sklearn.metrics import mean_squared_error
        3 mse = mean_squared_error(y_te, gbt_pred)
        4 print(mse)

22.210123556589096
```

AI/BigData 인텐시브 과정

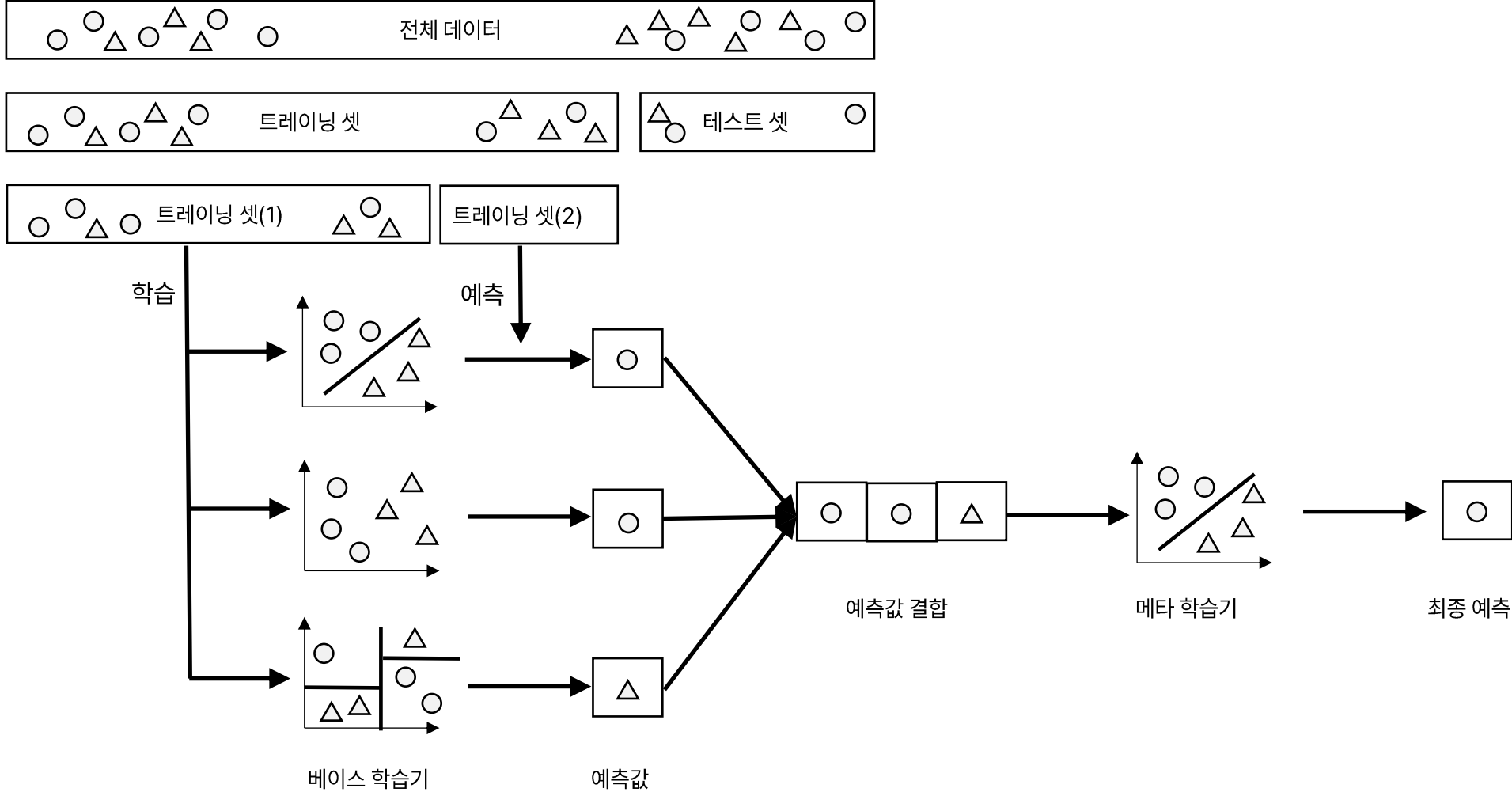
Section 4. 스택킹

Section 4-1. 스택킹 개념

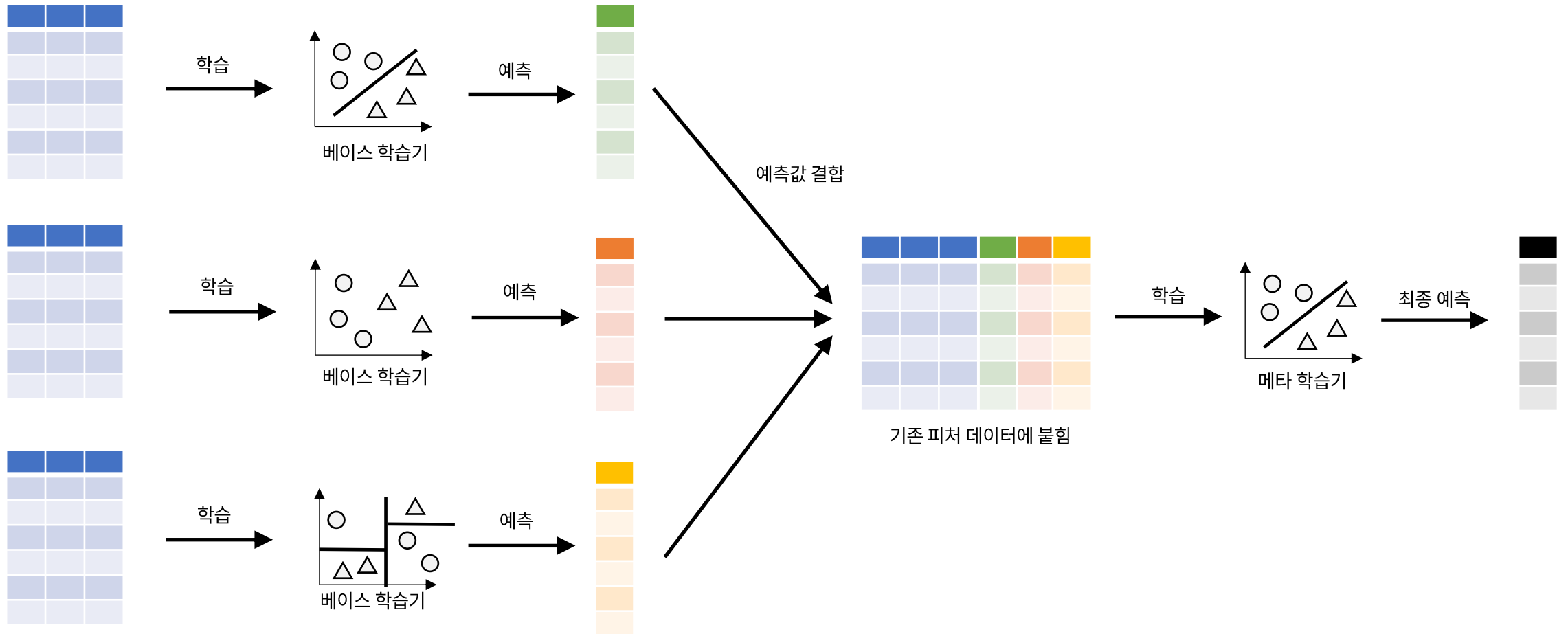
스태킹(stacking)

- 여러 가지 학습기를 쌓는 방법
- 스태킹 = 베이스 학습기(base learner)와 메타 학습기(meta learner)
- 베이스 학습기가 먼저 학습함
- 메타 학습기는 베이스 학습기의 예측을 피쳐 데이터로 활용해 최종 예측함

스태킹(stacking)



스태킹(stacking)



AI/BigData 인텐시브 과정

Section 4. 스택킹

Section 4-2. 스택킹 분류

스태킹(stacking) 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

스태킹(stacking) 분류

```
In [5]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

스태킹(stacking) 분류

```
In [6]: ▶ 1 # 트레이닝/테스트 데이터 분할  
2 from sklearn.model_selection import train_test_split  
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

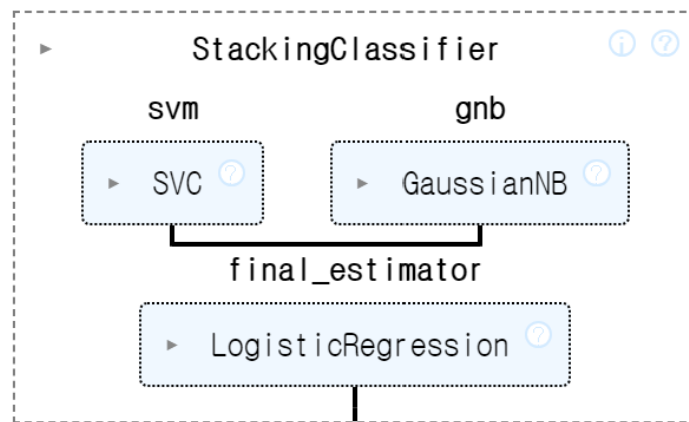
```
In [7]: ▶ 1 #데이터 표준화  
2 from sklearn.preprocessing import StandardScaler  
3 std_scale = StandardScaler()  
4 std_scale.fit(X_tn)  
5 X_tn_std = std_scale.transform(X_tn)  
6 X_te_std = std_scale.transform(X_te)
```

스태킹(stacking) 분류

In [6]: ▶

```
1 # 스태킹 학습
2 from sklearn import svm
3 from sklearn.naive_bayes import GaussianNB
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.ensemble import StackingClassifier
6
7 clf1 = svm.SVC(kernel='linear')
8 clf2 = GaussianNB()
9
10 clf_stkg = StackingClassifier(
11     estimators=[
12         ('svm', clf1),
13         ('gnb', clf2)
14     ],
15     final_estimator=LogisticRegression())
16 clf_stkg.fit(X_tn_std, y_tn)
```

Out [6]:



스태킹(stacking) 분류

In [7]: ▶

```
1 # 예측
2 pred_stkg = clf_stkg.predict(X_te_std)
3 print(pred_stkg)
```

```
[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 2 0 0 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```

스태킹(stacking) 분류

In [8]:



```
1 # 정확도(accuracy)
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, pred_stkg)
4 print(accuracy)
```

0.9555555555555556

In [9]:



```
1 # 정밀도(precision)
2 from sklearn.metrics import precision_score
3 precision = precision_score(y_te, pred_stkg, average='macro')
4 print(precision)
```

0.943355119825708

In [10]:



```
1 # 리콜
2 from sklearn.metrics import recall_score
3 recall = recall_score(y_te, pred_stkg, average='macro')
4 print(recall)
```

0.9682539682539683

In [11]:



```
1 # f1 스코어
2 from sklearn.metrics import f1_score
3 f1 = f1_score(y_te, pred_stkg, average='macro')
4 print(f1)
```

0.9536244800950683

스태킹(stacking) 분류

In [12]: ▶

```
1 # confusion matrix 확인
2 from sklearn.metrics import confusion_matrix
3 conf_matrix = confusion_matrix(y_te, pred_stkg)
4 print(conf_matrix)
```

```
[[16  0  0]
 [ 1 19  1]
 [ 0  0  8]]
```

In [13]: ▶

```
1 # 분류 레포트 확인
2 from sklearn.metrics import classification_report
3 class_report = classification_report(y_te, pred_stkg)
4 print(class_report)
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	16
1	1.00	0.90	0.95	21
2	0.89	1.00	0.94	8
accuracy			0.96	45
macro avg	0.94	0.97	0.95	45
weighted avg	0.96	0.96	0.96	45

AI/BigData 인텐시브 과정

Section 4. 스택킹

Section 4-3. 스택킹 회귀

스태킹(stacking) 회귀

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("./data/house_prices.csv")
```

```
In [2]: 1 df
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows x 14 columns

스태킹(stacking) 회귀

```
In [3]: ▶ 1 df['MEDV'].unique()
```

```
Out[3]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
                21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
                15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
                24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
                25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
                17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
                22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
                20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
                14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
                15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
                19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
                32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
                42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
                31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
                44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
                25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
                28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
                10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
                12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
                7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
                10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

스태킹 분류

```
In [4]: 1 df.columns
```

Out[4]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT'], dtype='object')

```
In [5]: 1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
2
3 X = df[features]
4 y = df['MEDV']
```

```
In [6]: 1 X
```

Out[6]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows x 13 columns

스태킹(stacking) 회귀

In [7]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [8]: ▶

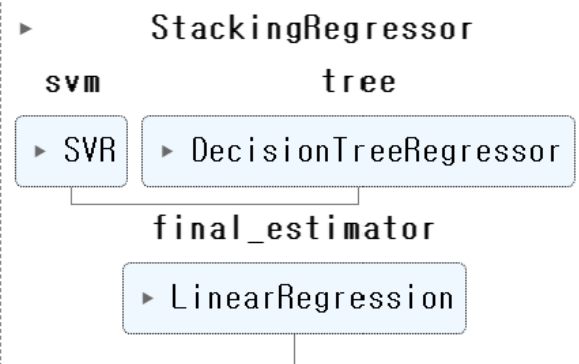
```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

스태킹(stacking) 회귀

In [13]: ▶

```
1 # 스태킹 학습
2 from sklearn.svm import SVR
3 from sklearn.tree import DecisionTreeRegressor
4 from sklearn.linear_model import LinearRegression
5 from sklearn.ensemble import StackingRegressor
6
7 clf1 = SVR(kernel='linear')
8 clf2 = DecisionTreeRegressor()
9
10 clf_stkg = StackingRegressor(
11     estimators=[
12         ('svm', clf1),
13         ('tree', clf2)
14     ],
15     final_estimator=LinearRegression())
16 clf_stkg.fit(X_tn_std, y_tn)
```

Out [13]:



스태킹(stackin

In [14]: ▶

```
1 # 예측
2 stkg_pred = clf_stkg.predict(X_te_std)
3 print(stkg_pred)
```

```
[24.34157089 23.23957348 24.59330003 13.82069655 21.16043766 20.02776052
 20.41659074 19.91458059 15.92084625 17.25659709  7.26451421 16.84821995
 14.4757953   6.69037582 45.54558369 35.46241721 21.14674882 35.41340329
 26.50832591 21.23571986 24.82889392 19.28921239 20.22448013 26.34873029
 22.52493611  7.77900739 17.67149541 17.83870988 38.62345701 19.13221042
 14.39324525 17.65132625 22.70398744 22.55941007 24.10707883 19.06772128
 10.6339768  21.93295338 13.87092063 16.77449268 24.67178441 23.10000463
 23.90792029 12.21485534 27.37998239 21.26565051 21.3737984  17.16861043
 12.31256245 27.19364452 16.07251082 19.24571836 23.59419818 35.08766368
 13.62666117 19.85281644 19.97896766 17.19788636  9.83452124 21.8283194
 20.00195371 22.12269626 30.95951282 26.83609775 20.43835381 27.21954542
 18.8441851  19.59942618 12.40979439 23.06362809 21.39705885 22.84658591
 26.29965839 30.02111638 24.87202092  6.21883318 37.83520614 23.50450054
 23.80361355 22.37943949 29.38871614 16.86901497 18.76779196 38.09602763
 41.45466628 24.80507394 22.02826242 13.62139696 25.0441948  15.28405241
 16.85687246 12.7461733  22.07181097 30.4046827  21.68699245 23.11923682
  5.70196283 24.1026607  14.84442066 17.5264938  25.31984394 21.00816568
 30.28858488 23.69335548 23.85398901 20.46026419  6.41959576 13.91595057
 23.25458491 24.75313805 34.79072995 15.51196985 19.46931629 19.8024878
 16.73495378 22.33325878  7.0918419  19.26162296  8.34355537 47.83968464
 30.2627386  9.04077141 15.60914613 21.2859308  23.29000037 20.3002156
 36.32747718]
```

In [15]: ▶

```
1 # 모델 평가-MSE
2 from sklearn.metrics import mean_squared_error
3 mse = mean_squared_error(y_te, stkg_pred)
4 print(mse)
```

27.471756652170892

감사합니다.

Q & A